VEXEN

CONTROL AI.

The permission layer for AI agents.

One API call answers **“is this specific agent allowed to do this specific thing, right now?”** — ALLOW, BLOCK, or ASK A HUMAN. And it actually stops the action when the answer is no.

STAGE	USERS	BUILT	TEAM	CATEGORY
Live · pre-revenue	5,000+	Solo, in 2 months	1 founder	Agent infrastructure

We gave AI hands and forgot the keys.

UNTIL RECENTLY

AI wrote text. A human decided.

The model produced words, a person read them, and the person chose whether to act. The blast radius of a mistake was **embarrassment**.

TODAY

AI holds credentials. AI decides.

Agents act without a human in the loop. The blast radius is now **your bank balance, your customer relationships and your compliance posture**.

PLACES ORDERS | MOVES MONEY | ISSUES REFUNDS | EMAILS CUSTOMERS | WRITES TO PRODUCTION | BOOKS TRAVEL | DELETES FILES

Agents now do all of this with an employee-scoped credential.

A prompt is not a permission.

The typical production agent holds a long-lived credential with roughly an employee's scope, because narrowing it was fiddly and nobody had time. Its limits live in the prompt, where they are advisory.

```
SYSTEM_PROMPT = """
You are a purchasing agent.
Never spend more than $100.
Always ask before refunding.
"""

# ↑ these are suggestions.
# nothing enforces them.
# nothing records the attempt.
```

FAILURE 1 — MOST COMMON

The confident mistake

Not attacked, not jailbroken — just wrong. Buys 500 units instead of 5 because a price was per-case. Refunds twice because the first had not settled. Almost entirely preventable with limits.

FAILURE 2 — ADVERSARIAL

The manipulated agent

Indirect prompt injection hides in a page, a PDF, a support ticket. Every published model-layer defence has been bypassed. You cannot fix this inside the thing being attacked.

FAILURE 3 — WHY PILOTS STALL

The accountability vacuum

What may this agent do, right now? What was refused? Who approved the exception, and can you prove the record is unedited? No answers means no production deployment.

Every computing generation grows a permission layer. This one hasn't.

MAINFRAME	NETWORK	CLOUD	SAAS IDENTITY	AI AGENTS
RACF	Active Directory	IAM	Okta	nothing

TRIGGER 1

Tool use got reliable

Teams now trust agents with real side effects, not drafts.

TRIGGER 2

Loops run unattended

Every major framework shipped agent loops with no human in the middle.

TRIGGER 3

The operator changed

Ops leads and finance managers now deploy agents — and carry the blame.

> The gap opened about a year ago and widens every week as more agents get more keys. Permission layers get chosen once, and then they stay.

Four categories point at this. None of them stop anything.

CATEGORY	WHAT IT ACTUALLY DOES	WHY IT DOES NOT SOLVE THIS
Cloud IAM AWS · GCP · Okta · Entra	Authorises a principal against a resource, statically.	No per-action amount, no human-in-the-loop primitive, no approval that expires. “May call the Stripe API” is not “may refund up to $50”.
Guardrails filters · moderation · PII	Inspects text: toxic, off-topic, leaking, off-brand.	Says nothing about whether an action is permitted. A polite, PII-clean message that wires $40,000 to the wrong account passes every one.
Observability tracing · evals	Records what the agent did, afterwards.	Read-only and after the fact. It is the flight recorder, not the flight control. A complement, not a competitor.
In-framework HITL interrupts	Pauses the agent from inside the agent process.	The agent decides when to ask. The pause dies with the process. No record, no audit trail, no non-technical approver. Self-policing restated.

> **The gap:** nothing external, deterministic and per-action that a non-engineer can configure, a non-engineer can approve from, that genuinely halts the action — and that produces evidence.

The actor does not get to decide the limits of its own authority.

Banks, hospitals, aviation and payment processors all separate three things that agent deployments currently fuse into a single model call.

1 — IDENTITY

Who is asking

The agent, with its own key. Never shared, never ambiguous in the record.

2 — POLICY

What they may do

Decided **elsewhere, by someone else**, in a workspace the agent cannot read or write. This is the piece that is missing today.

3 — EXECUTION

What happens

The side effect — but only after policy has answered.

> A bank teller can be completely honest and still not be permitted to move ten million dollars alone. At scale you cannot verify intent. You can only **constrain consequence**.

The teller does not set their own withdrawal limit. The pilot does not grant their own clearance.

One call. Three answers.

```
POST /api/v1/authorize
X-Vexen-Key: <the agent's own key>

{
  "resource":    "shopping",
  "action":      "purchase",
  "amount":      742.50,
  "currency":    "USD",
  "resource_id": "order-4471"
}
```

ALLOW

Proceed. Recorded with the full evaluation trace.

APPROVAL_REQUIRED

Stop. A named human has been asked, and here is the approval id.

BLOCK

Refused, with a reason in language the operator can act on.

MACHINE-READABLE REASON CODE

HUMAN-READABLE REASON

FULL 13-STEP TRACE

PER-AGENT HASHED KEYS

Deterministic by design. No model in the enforcement path.

A pure function: no I/O, no inference. Thirteen ordered steps, each one recorded in the trace.

01 Verify agent identity	02 Verify agent is active	03 Verify request is authenticated	04 Validate request shape	05 Permission exists at all
06 Permission not expired	07 Resource restrictions	08 Action restrictions	09 Numeric limits	10 Sensitive-resource rules
11 Workspace policy	12 Human-approval requirement	13 Calculate risk → decision	↳ the trace *is* the explanation	↳ console, audit & SDK error agree

Default-deny. The only path to ALLOW is an explicit grant that survives all thirteen. Same input, same answer, every time — the only kind of gate you can certify.

The AI writes rules. The AI never enforces them.

You type “any purchase over $100 needs a person to approve it”; a model turns it into a schema-validated rule. You cannot certify a gate that might answer differently tomorrow.

Approvals that actually hold.

Most human-in-the-loop implementations get one or two of these five. Missing any single one gives you an approval that can be replayed, escalated or forged — **worse than no approval at all**, because now a signed-looking record says a human agreed.

01

Single-use

Consumed the moment it is spent. No replay, ever.

02

Short-lived

30 minutes by default. Yesterday's yes authorises nothing today.

03

Atomically claimed

Two concurrent requests can never both spend it.

04

Scope-bound

Same permission, same order, same amount **to the cent**.

05

Zero client trust

State loads server-side. Caller claims are ignored.

`VERIFIED: REPLAY REFUSED` `VERIFIED: AMOUNT ESCALATION REFUSED` `VERIFIED: CROSS-ORDER REUSE REFUSED` `VERIFIED: FORGED TOKEN REFUSED`

Two taps. No login. Fifteen seconds.

A control humans find annoying gets switched off, and a switched-off control protects nobody. So convenience is treated as a security requirement, not a nicety.

THE SUBTLETY THAT SHAPED THE DESIGN

A link that acts on GET is a vulnerability.

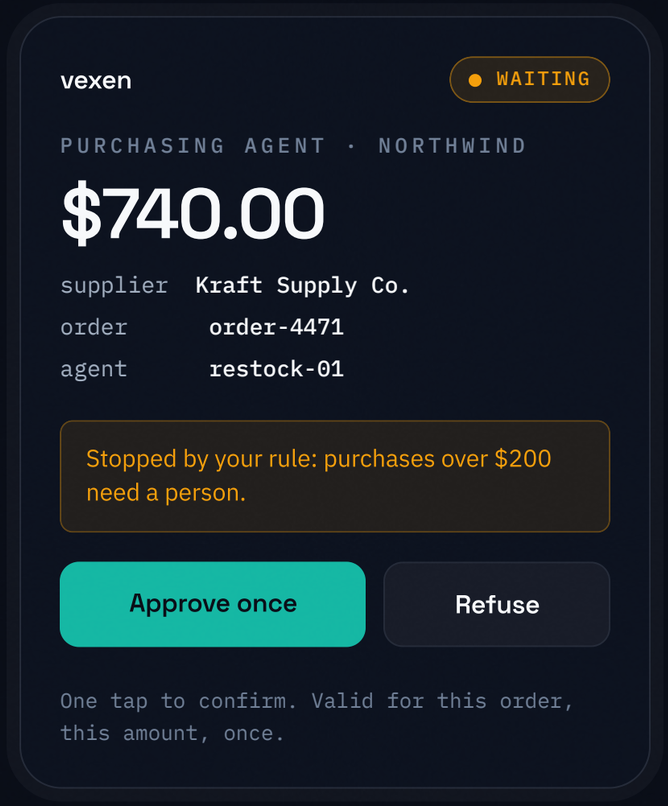
Mail providers and corporate link scanners fetch every URL in a message to check it. If Approve happened on GET, **your security scanner would approve the $40,000 purchase before you opened the email**. So the GET is strictly read-only and the decision requires a deliberate POST.

HMAC-SHA256, DOMAIN-SEPARATED

IDS ONLY — NEVER A SESSION

24-HOUR EXPIRY

MINTED PER RECIPIENT



A record you can hand to someone who does not trust you.

DECISION	AGENT	ACTION	AMOUNT	APPROVED BY	INTEGRITY
ALLOW	restock-01	shopping.purchase	$182.00	— rule	✓ 9f2c…a41b
APPROVED	restock-01	shopping.purchase	$740.00	a.morgan	✓ 3d81…c07e
BLOCK	support-02	data.export	—	— policy	✓ 77ae…1f93
BLOCK	restock-01	payments.credential	—	— policy	✓ b520…8ddc

SHA-256 over every event's identifying fields. A verification pass recomputes each hash and names the rows that changed, if any.

So "our agent did not do that" and "a human approved this on the 14th" become **checkable claims** rather than assertions. This is the feature that comes up first in every serious conversation.

vexen

Governed in under a minute.

Governance software dies at the blank page. Nobody knows which of 67 permissions their shopping agent needs, so they grant everything — and **false assurance is worse than none**.

So the first experience is a correctly-configured agent you can then relax, not an empty form you will fill in wrong. Defaults are the product, and this is the single biggest lever on activation.

READY-MADE AGENT CONFIGURATIONS

10,639

55 archetypes × 28 industries × 7 capability tiers

SEARCH “REFUND”

DEPLOY IN ONE CLICK

NARROWED · CAPPED · THRESHOLDED

67
permissions in business language, not endpoints

18
categories, from Payments to Healthcare

10
flagged sensitive, with stricter defaults

4
server-side roles; approvers cannot edit policy

$740 of packaging, at three in the morning.

A three-person e-commerce company. The purchasing agent holds a payment key and a supplier login.

BEFORE

Nobody is checking

The agent does whatever the model decides overnight. The founder finds out from her bank.

SETUP — 60 SECONDS

Deploy & one rule

One click from the library: up to $200, three approved suppliers, $2,000 daily cap. She types “never let this agent touch payment credentials”.

INTEGRATION — ONE LINE

Guard the function

Her engineer adds a decorator above the function that actually places the order. If Vexen says no, it never runs.

3:04AM

Halted, then allowed once

$740 attempt stops. Her phone asks. She taps approve. It goes through — once, that amount, that order. Replay tomorrow is refused.

Nothing in that story requires her to understand an API key, and nothing in it requires her to **trust** the agent. Two weeks later her accountant asks about the $740, and she hands over a record with a hash proving nobody edited it.

Built solo in two months. Live, and past five thousand users.

USERS

5,000+

no paid acquisition, content-led

BUILD TIME

2 mo

one person, end to end, deployed

REGRESSION SUITE

55/55

auth, tenant isolation, replay refusal, role guards

INDEPENDENT PASS

110/110

backend checks, no critical defects

WHAT USERS KEEP ASKING FOR

TypeScript · native framework guards · **“can I stop the agent bypassing this at all?”**

That last one is a request for the credential broker. Users are pulling the product toward the harder, more defensible version of itself.

NOT DONE — STATED PLAINLY

No revenue yet. Python-only SDK. One workspace per account. No SSO, SCIM or SOC 2. The credential broker is designed but unbuilt.

A security product that overstates its readiness is the exact thing we argue against.

Bought once. Removed almost never.

Approver seats plus decision volume, because value scales with the number of consequential decisions we sit in front of.

TIER	WHO, AND WHAT UNLOCKS
Free	One workspace, monthly decision allowance, **full audit trail**. The acquisition engine.
Team	Per approver seat + volume. First revenue; the initial wedge.
Business	SSO, retention, policy bundles, residency. Real budget.
Platform	API-metered, white-label. Volume without headcount.

BOTTOM-UP, 2028 — OUR ESTIMATE, NOT AN ANALYST REPORT

~50,000 companies × ~$25k ACV ≈ **$1.25B**

If permissions become table stakes rather than a choice — the way nobody ships a cloud workload without IAM — the population is ~500,000 and the market is **$10B+**.

ANCHORS

IAM is a multi-tens-of-billions market; GRC tooling is larger. Agent authorization is the agent-native slice of two existing budgets — so we never have to argue the category deserves money.

Zero I/O and no model call in the decision path. Marginal cost per decision is near nil — gross margin looks like infrastructure, not an LLM wrapper.

Sixth product. Over 100,000 users before this one.

Every one built, shipped and distributed with no marketing budget. Distribution is what most technical founders lack; it is the thing I have repeatedly done.

PRODUCT	USERS	WHAT IT DOES, AND WHY IT IS RELEVANT
HealthLens	70,000+	AI wellness platform — health data in one place, food tracking, facial and voice analysis.
AI-Namo	20,000+	AI government-document verification, **adopted by a local government** for fraud prevention — an institution depending on my software to make a trust decision.
GreenGuide	10,000+	Agricultural copilot: plant health diagnosis, variety ID, contamination risk.
CerebroAegis	5,000+	Health and safety platform turning complex guidance into actionable steps.
The1550Formula	~$70k/mo	COO of an AI-first test-prep company, 1,000+ students, ~200 point average gain — where I learned the distance between a signup and a customer.

THE THROUGH-LINE

Every product is an interpreter between a person and complex information generated about them. Vexen interprets an agent's intended action for the person accountable — then does what none of the others could: **stops it**.

WHERE I AM WEAK

I have never sold an infrastructure security product through an enterprise procurement cycle. That is hire #1, and I would rather name it than have you find it.

Raising a pre-seed to make bypassing impossible.

Today the guard is a check an agent chooses to make. The complete version brokers short-lived, scoped credentials, so an agent **physically cannot** exceed its authority.

USE OF FUNDS — 1

Two engineers

The credential broker, a TypeScript SDK, and native guards for the main agent frameworks. Ideally out of IAM, fraud or payments.

USE OF FUNDS — 2

First GTM hire

Someone who has sold an infrastructure control to a mid-market ops or finance buyer. The honest gap in my skill set.

USE OF FUNDS — 3

Trust & compliance

External security review — a security product never attacked by a stranger is an untested claim — then SOC 2, SSO and SCIM.

FIRST 90 DAYS

1–4: sit with ten teams running agents in production and watch them configure it unaided. Ship the TypeScript SDK.
5–8: multi-workspace, framework guards, begin the external security review — and publish what it finds.
9–12: credential broker v1 live with a design partner. Start SOC 2 Type I.

WHAT I WANT BEYOND CAPITAL

Two or three introductions to ops or finance leaders inside portfolio companies already running agents in production.

Not necessarily as customers — as the people who will tell me in ten minutes whether the approval flow survives a real approval chain.

Agents already have the keys.
Somebody has to hold the lock.

ALLOW · APPROVAL_REQUIRED · BLOCK

TRY IT

Block a live agent, free

WATCH IT

Two-minute walkthrough

READ IT

19-page technical document

BREAK IT

Try to replay an approval

The fastest evaluation: deploy an agent, set the threshold to $10, then try to make it spend $50.